LABORATORY EXERCISE 6

Student Portal Management System

Inventory workflow adapted to course enrollment

Student ID: 2411600038

Last name: Agan

Implementation report and evidence

Project Summary

This Laravel application adapts the Lab 6 inventory workflow to a Student Portal. Course capacity is treated as available stock; enrollment, cancellation, and adjustment actions create a persistent transaction history and update the related seat count.

Lab 6 concept -> Student Portal equivalent

Product / quantity -> Course / available slots

Stock movement -> Enrollment, cancellation, slot adjustment

Inventory transaction -> EnrollmentTransaction linked to course and user

Low-stock alert -> Few-slots notification

Inventory dashboard / reports -> Portal statistics, enrollment and availability reports

Verification: 15 tests passed, 96 assertions; migrations are applied. Dashboard data refreshes every 30 seconds.

Database evidence: the application uses SQLite. Artisan output is included instead of phpMyAdmin because phpMyAdmin manages MySQL and cannot inspect this SQLite database. Existing local MySQL databases were not changed.

GitHub status: repository screenshot and push are pending a destination URL. No remote was guessed and no push was attempted.

Lab 5 Foundation and Connection to Lab 6

Laboratory Exercise 5 established the foundation of the Student Portal using Laravel's MVC architecture. The system used Models to manage data and database logic, Views for the user interface, and Controllers to process user requests. It also implemented Laravel features such as Eloquent ORM, Blade templates, migrations, routing, validation, authentication, and Artisan commands. The application was connected to a database and supported CRUD operations and a dashboard using real data. These features served as the foundation for Laboratory Exercise 6, where the system was extended with transaction tracking, automatic updates, notifications, real-time dashboard features, and other advanced functionality.

LAB 5 FOUNDATION

Laravel MVC -> Eloquent and database -> CRUD -> Authentication -> Real-data dashboard

LAB 6 EXTENSION

Transactions -> Automatic updates -> Notifications -> API / AJAX dashboard -> Reports and history

Lab 5 established the application structure and server-side data foundation. Lab 6 builds on that same MVC and database workflow by adding coordinated business events: each enrollment transaction automatically changes course availability, creates an auditable history entry, and can trigger notifications and refreshed dashboard statistics.

Questions and Answers

Question 1 - What should happen if updating the related data fails after a transaction has been created?

Both the transaction record and the course slot update should be performed inside the same DB::transaction() block. If the slot update fails, the database rolls back the transaction insert too, preventing a history record that claims an enrollment occurred when availability did not change (or the reverse).

Question 2 - What happens when an exception occurs inside DB::transaction()?

Laravel rolls back the database transaction automatically and then rethrows the exception. The application can catch the error at the appropriate service or controller boundary to return a clear failure response. The rollback ensures the database is not left with only some of the related writes.

Question 3 - AJAX polling versus WebSockets

AJAX polling is simpler to implement and suits periodic updates. This portal polls its dashboard API every 30 seconds, which is easy to operate but can show data up to one interval old and causes recurring requests. WebSockets keep a connection open so the server can push updates immediately, providing more real-time behavior and avoiding repeated polling requests; they require more complex server, connection, and deployment support. For this Lab 6 dashboard, polling is an appropriate trade-off.

Conclusion (250 words)

This laboratory exercise demonstrates that inventory-management principles can be adapted to a student portal by treating course seats as limited stock and enrollments as recorded transactions. Courses store their maximum capacity, current availability, and low-slot threshold. Students can browse courses and enroll or cancel, while administrators can manage courses and perform reasoned slot adjustments. Each action is recorded with its related course and user, transaction type, quantity, reference, notes, and timestamp. An observer keeps enrollment records and course availability synchronized, and database transactions protect the application from partial updates. Validation and authorization prevent duplicate or invalid enrollment activity, negative capacity, overbooking, and unauthorized management actions.

The portal also provides enrollment history, searchable reports, few-slot notifications, and a dashboard that displays student, course, enrollment, and capacity statistics. Its dashboard API is polled every thirty seconds, which makes the displayed totals refresh without requiring a full page reload. The test suite passed all fifteen tests and ninety-six assertions during preparation of this report. Migration status confirms the course, enrollment, notification, and authorization schema is installed. This project uses SQLite locally, so its database was verified with Laravel Artisan rather than phpMyAdmin, which manages MySQL databases. The code and screenshots are prepared locally; publishing to GitHub and capturing that repository page remain pending because a destination repository URL has not been provided. Overall, the implementation preserves the central Lab 6 lesson: related data changes must be validated, applied atomically, and recorded so that administrators can review activity and students receive accurate availability information.

Screenshot 1: Course model

Course fields, relationships, availability helpers, and enrollment status.

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Attributes\Fillable;
use Illuminate\Database\Eloquent\Builder;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasMany;

#[Fillable(['code', 'name', 'category', 'instructor', 'description', 'maximum_slots', 'available_slots',
'minimum_slots'])]
class Course extends Model
{
    /**
     * @return HasMany<EnrollmentTransaction, $this>
     */
    public function enrollmentTransactions(): HasMany
    {
        return $this->hasMany(EnrollmentTransaction::class);
    }

    /**
     * @return HasMany<Enrollment, $this>
     */
    public function enrollments(): HasMany
    {
        return $this->hasMany(Enrollment::class);
    }

    public function hasAvailableSlots(): bool
    {
        return $this->available_slots > 0;
    }

    public function isAlmostFull(): bool
    {
        return $this->hasAvailableSlots()
            && $this->available_slots <= $this->minimum_slots;
    }

    public function isFull(): bool
    {
        return ! $this->hasAvailableSlots();
    }

    public function getEnrollmentStatusAttribute(): string
    {
        return $this->getEnrollmentStatus();
    }

    public function getEnrollmentStatus(): string
    {
        if ($this->isFull()) {
            return 'Full';
        }

        return $this->isAlmostFull() ? 'Almost Full' : 'Available';
    }

    /**
     * @param  Builder<Course>  $query
     * @return Builder<Course>
     */
    public function scopeAlmostFull(Builder $query): Builder
    {
        return $query->where('available_slots', '>', 0)
            ->whereColumn('available_slots', '<=', 'minimum_slots');
    }

    /**
     * @param  Builder<Course>  $query
     * @return Builder<Course>
     */
    public function scopeFull(Builder $query): Builder
    {
        return $query->where('available_slots', '<=', 0);
    }

    /**
     * @param  Builder<Course>  $query
     * @return Builder<Course>
     */
    public function scopeAvailable(Builder $query): Builder
    {
        return $query->where('available_slots', '>', 0);
    }
}
```

Screenshot 2: Enrollment transaction controller

Controller implementation for enrollment transaction actions.

```
<?php

namespace App\Http\Controllers;

use App\Models\Course;
use App\Models\EnrollmentTransaction;
use App\Services\EnrollmentTransactionService;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\View\View;

class EnrollmentTransactionController extends Controller
{
    public function index(Request $request): View
    {
        $transactions = EnrollmentTransaction::query()
            ->with(['course', 'user'])
            ->when(! $request->user()->isAdmin(), fn ($query) => $query->where('user_id', $request->user()->id))
            ->when($request->filled('type'), fn ($query) => $query->where('type', $request->string('type')-
>toString()))
            ->latest()
            ->paginate(15)
            ->withQueryString();

        return view('enrollments.index', compact('transactions'));
    }

    public function create(): View
    {
        return view('enrollments.create', [
            'courses' => Course::available()->orderBy('name')->get(),
        ]);
    }

    public function store(Request $request, EnrollmentTransactionService $service): RedirectResponse
    {
        $validated = $request->validate([
            'course_id' => ['required', 'integer', 'exists:courses,id'],
        ]);
        $course = Course::findOrFail($validated['course_id']);

        $transaction = $service->enroll($course, $request->user());

        return redirect()
            ->route('enrollments.show', $transaction)
            ->with('success', 'You are enrolled. One available slot has been reserved.');
    }

    public function show(EnrollmentTransaction $enrollmentTransaction): View
    {
        abort_unless(
            auth()->user()->isAdmin() || $enrollmentTransaction->user_id === auth()->id(),
            403,
        );

        return view('enrollments.show', [
            'transaction' => $enrollmentTransaction->load(['course', 'user']),
            'isEnrolled' => $enrollmentTransaction->course
                ->enrollments()
                ->where('user_id', $enrollmentTransaction->user_id)
                ->exists(),
        ]);
    }

    public function cancel(
        Course $course,
        Request $request,
        EnrollmentTransactionService $service,
    ): RedirectResponse {
        $service->cancel($course, $request->user());

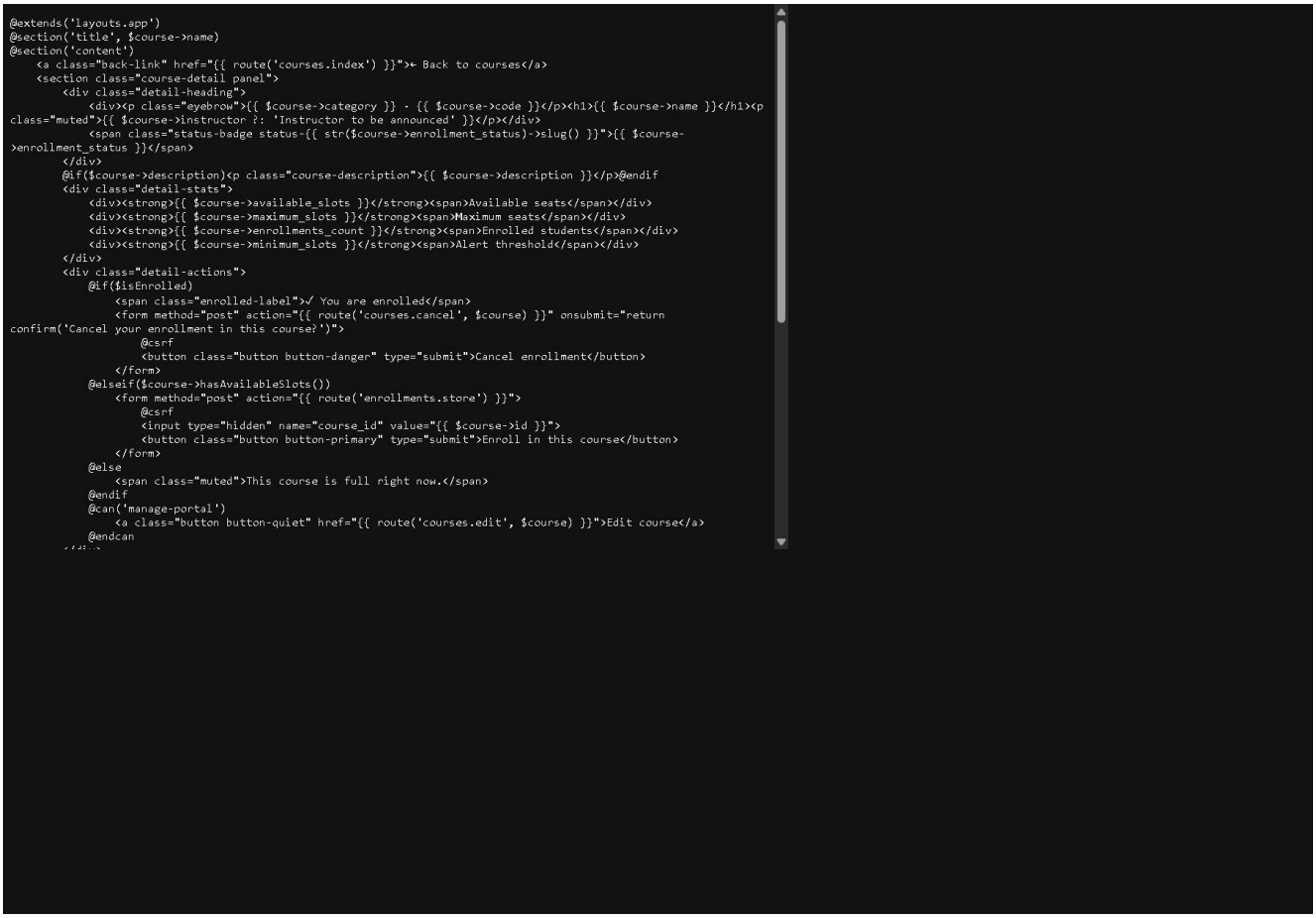
        return redirect()->route('courses.show', $course)->with('success', 'Enrollment cancelled and one slot
restored.');
    }

    public function destroy(EnrollmentTransaction $enrollmentTransaction): RedirectResponse
    {
        abort_unless(auth()->user()->isAdmin(), 403);

        return back()->with(
            'error',
            'Enrollment transactions are kept as an audit history. Record a cancellation or adjustment instead of
deleting one.',
        );
```

Screenshot 3: Blade view template

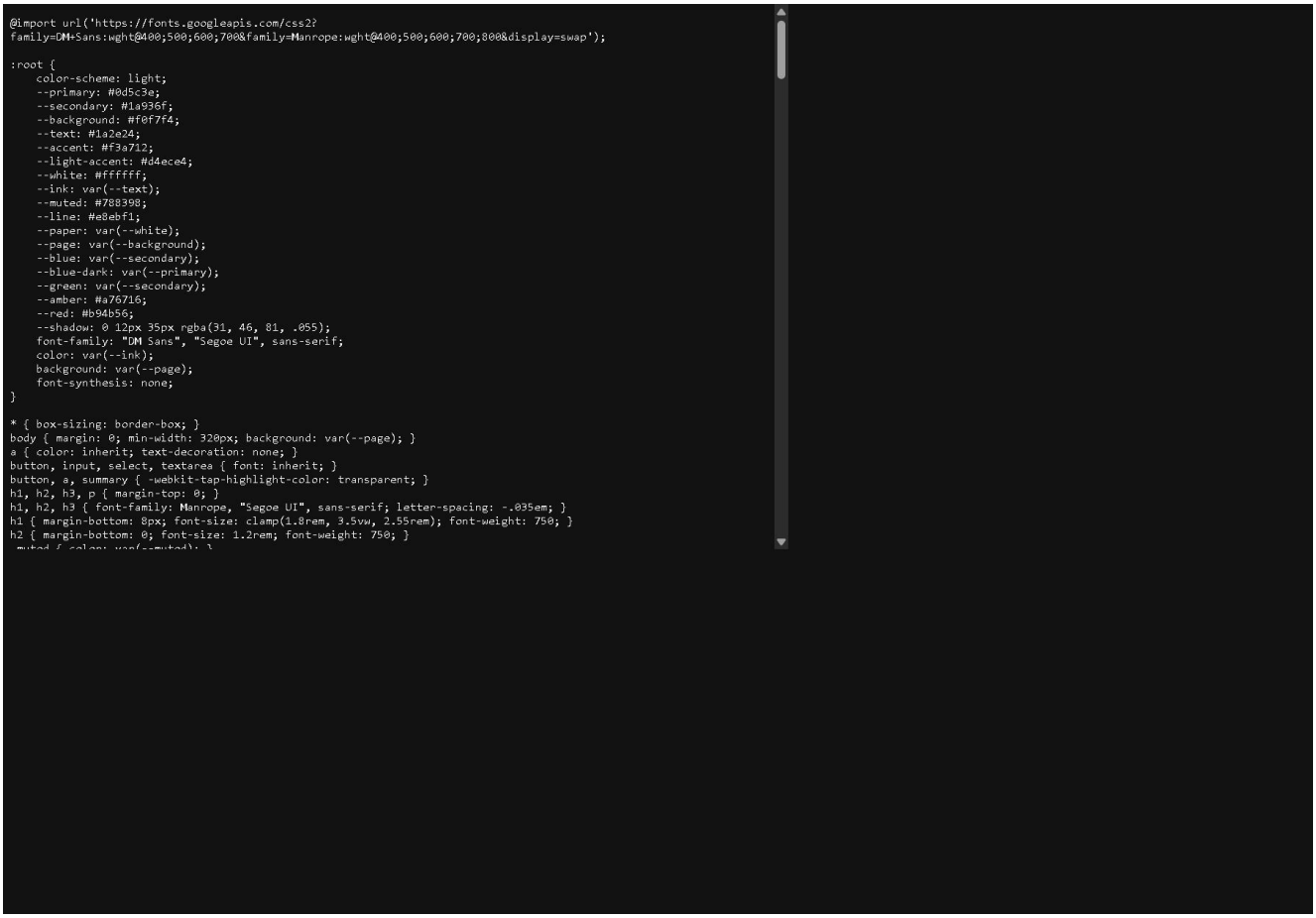
Course detail page template.



```
@extends('layouts.app')
@section('title', $course->name)
@section('content')
    <a class="back-link" href="{{ route('courses.index') }}">← Back to courses</a>
    <section class="course-detail panel">
        <div class="detail-heading">
            <div><p class="eyebrow">{{ $course->category }} · {{ $course->code }}</p><h1>{{ $course->name }}</h1><p
class="muted">{{ $course->instructor ?: 'Instructor to be announced' }}</p></div>
            <span class="status-badge status-{{ str($course->enrollment_status)->slug() }}">{{ $course-
>enrollment_status }}</span>
        </div>
        @if($course->description)<p class="course-description">{{ $course->description }}</p>@endif
        <div class="detail-stats">
            <div><strong>{{ $course->available_slots }}</strong><span>Available seats</span></div>
            <div><strong>{{ $course->maximum_slots }}</strong><span>Maximum seats</span></div>
            <div><strong>{{ $course->enrollments_count }}</strong><span>Enrolled students</span></div>
            <div><strong>{{ $course->minimum_slots }}</strong><span>Alert threshold</span></div>
        </div>
        <div class="detail-actions">
            @if($isEnrolled)
                <span class="enrolled-label">✓ You are enrolled</span>
                <form method="post" action="{{ route('courses.cancel', $course) }}" onsubmit="return
confirm('Cancel your enrollment in this course?')">
                    @csrf
                    <button class="button button-danger" type="submit">Cancel enrollment</button>
                </form>
            @elseif($course->hasAvailableSlots())
                <form method="post" action="{{ route('enrollments.store') }}">
                    @csrf
                    <input type="hidden" name="course_id" value="{{ $course->id }}">
                    <button class="button button-primary" type="submit">Enroll in this course</button>
                </form>
            @else
                <span class="muted">This course is full right now.</span>
            @endif
            @can('manage-portal')
                <a class="button button-quiet" href="{{ route('courses.edit', $course) }}">Edit course</a>
            @endcan
        </div>
```

Screenshot 4: Custom CSS

Requested green color palette and shared styling.



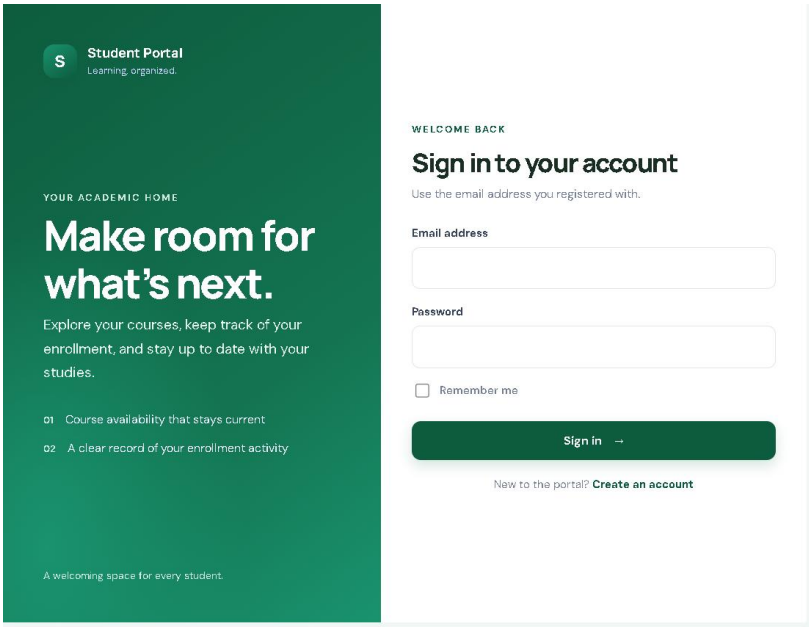
```
@import url('https://fonts.googleapis.com/css2?
family=DM+Sans:wght@400;500;600;700&family=Manrope:wght@400;500;600;700;800&display=swap');

:root {
    color-scheme: light;
    --primary: #0d5c3e;
    --secondary: #1a936f;
    --background: #f0f7f4;
    --text: #1a2e24;
    --accent: #f3a712;
    --light-accent: #d4ece4;
    --white: #ffffff;
    --ink: var(--text);
    --muted: #788398;
    --line: #e8ebf1;
    --paper: var(--white);
    --page: var(--background);
    --blue: var(--secondary);
    --blue-dark: var(--primary);
    --green: var(--secondary);
    --amber: #a76716;
    --red: #b94b56;
    --shadow: 0 12px 35px rgba(31, 46, 81, .055);
    font-family: "DM Sans", "Segoe UI", sans-serif;
    color: var(--ink);
    background: var(--page);
    font-synthesis: none;
}

* { box-sizing: border-box; }
body { margin: 0; min-width: 320px; background: var(--page); }
a { color: inherit; text-decoration: none; }
button, input, select, textarea { font: inherit; }
button, a, summary { -webkit-tap-highlight-color: transparent; }
h1, h2, h3, p { margin-top: 0; }
h1, h2, h3 { font-family: Manrope, "Segoe UI", sans-serif; letter-spacing: -.035em; }
h1 { margin-bottom: 8px; font-size: clamp(1.8rem, 3.5vw, 2.55rem); font-weight: 750; }
h2 { margin-bottom: 0; font-size: 1.2rem; font-weight: 750; }
.muted { color: var(--muted); }
```

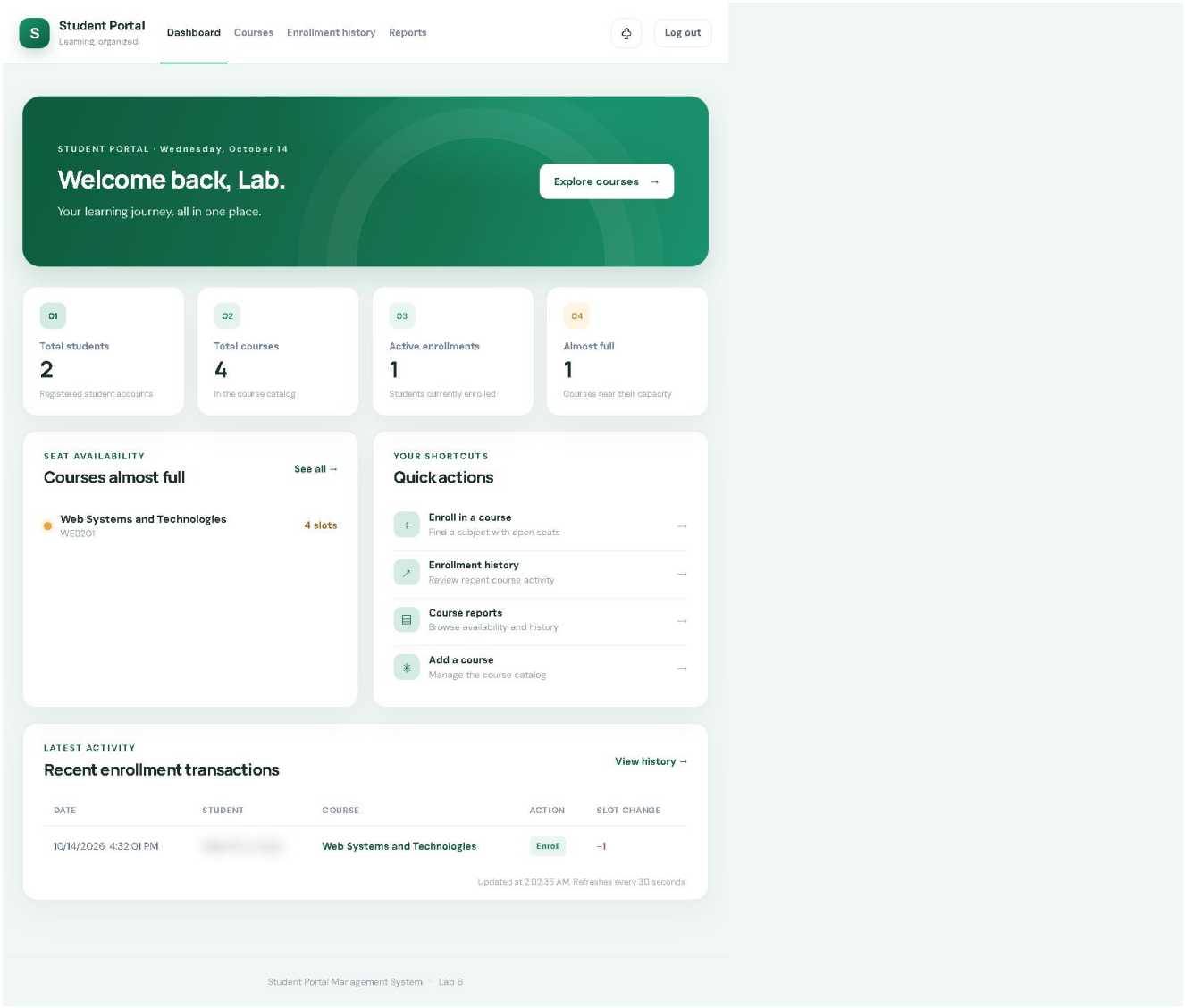
Screenshot 5: Login page

Student Portal sign-in page.



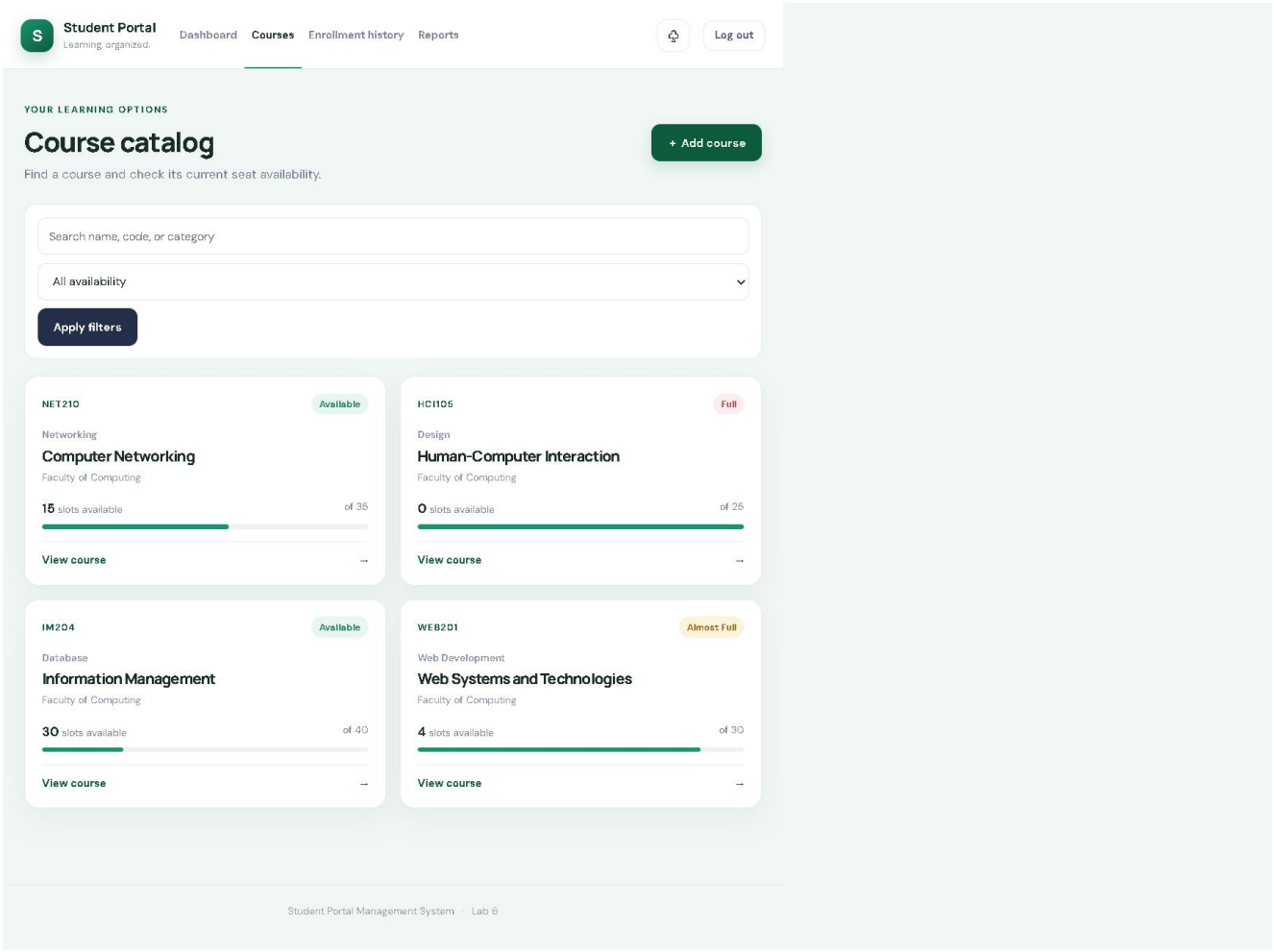
Screenshot 6: Dashboard statistics

Live portal statistics and recent activity; student identity is blurred for privacy.



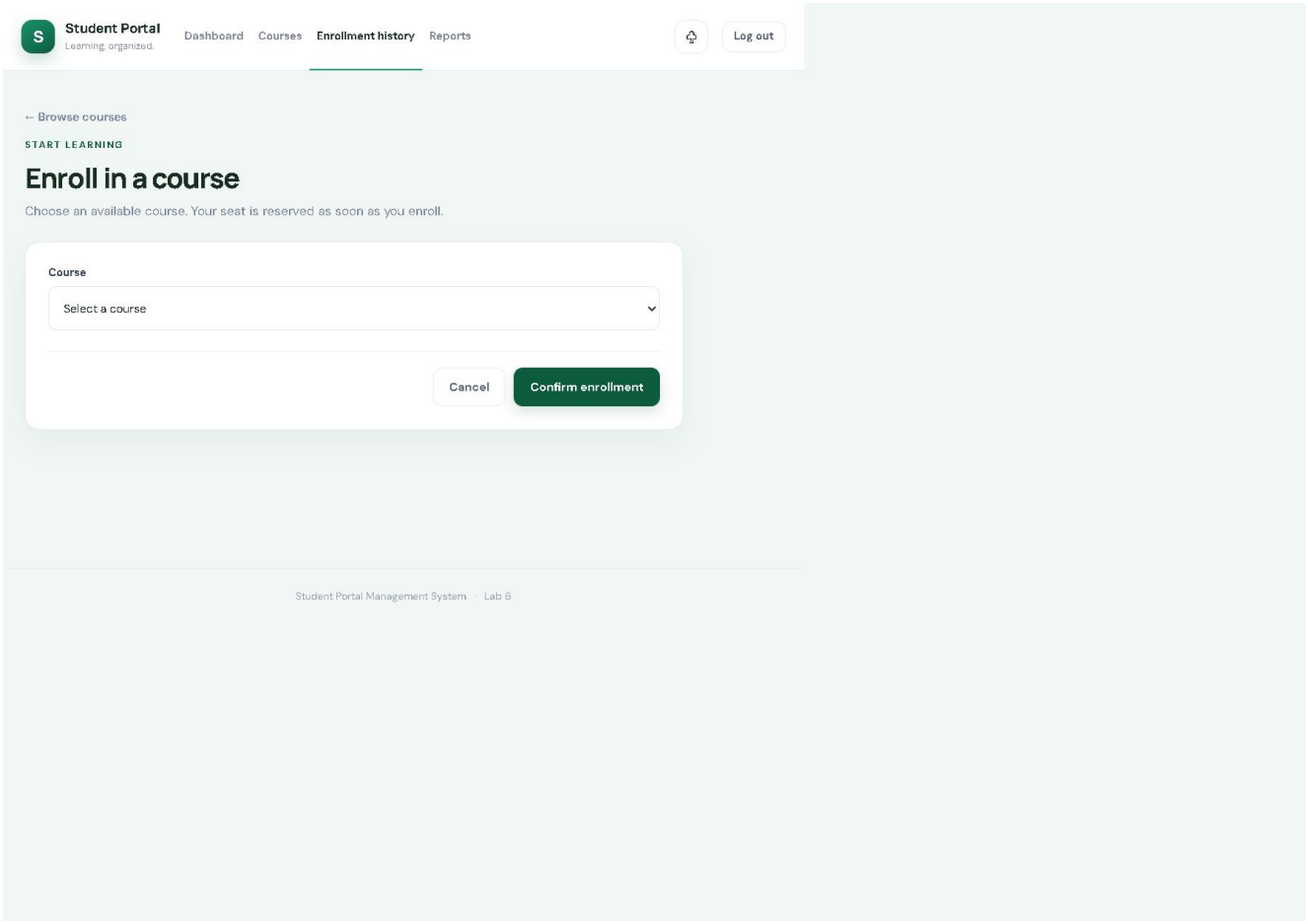
Screenshot 7: Course index

Course catalog with availability statuses and filters.



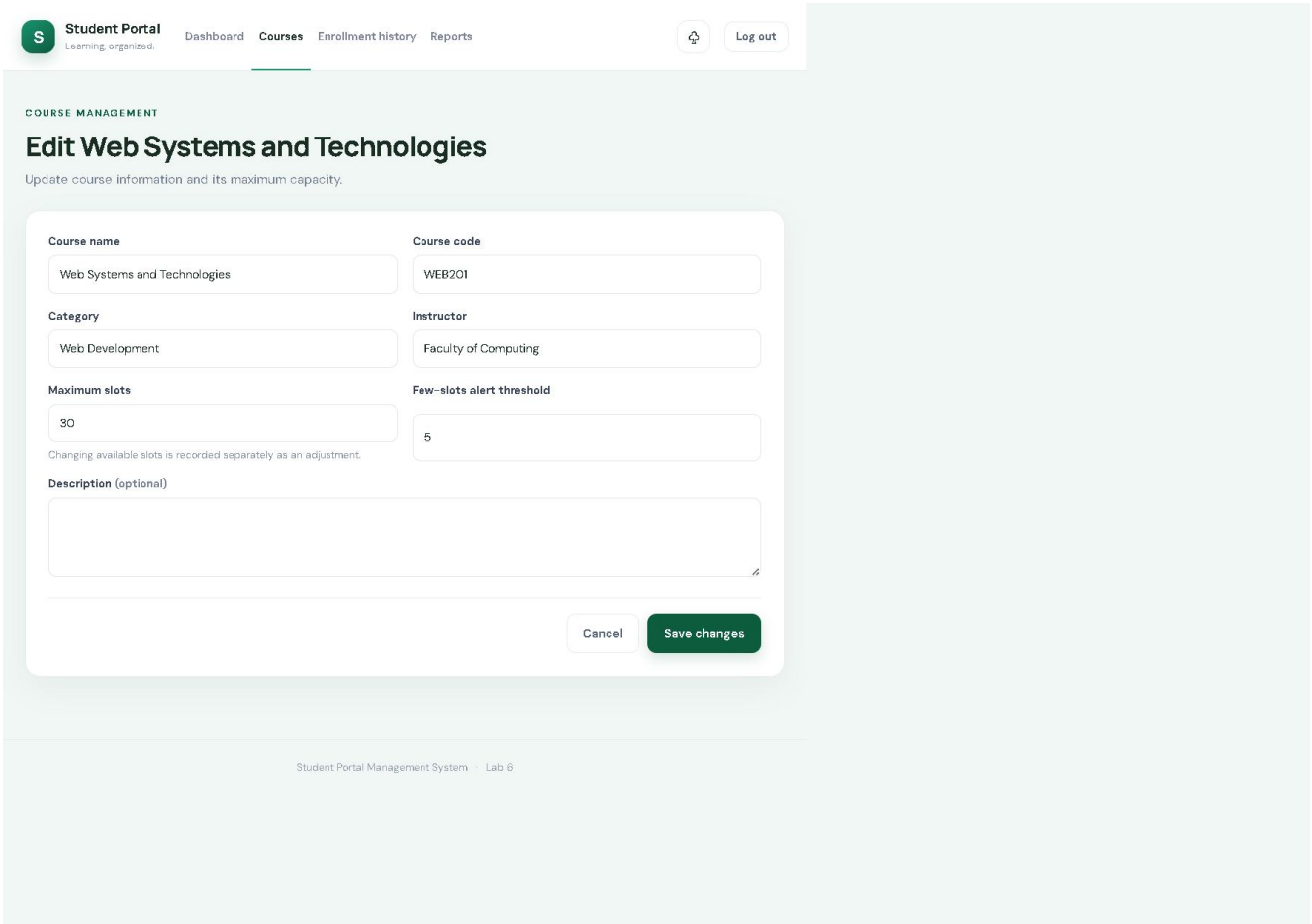
Screenshot 8: Enrollment form

Student enrollment form.



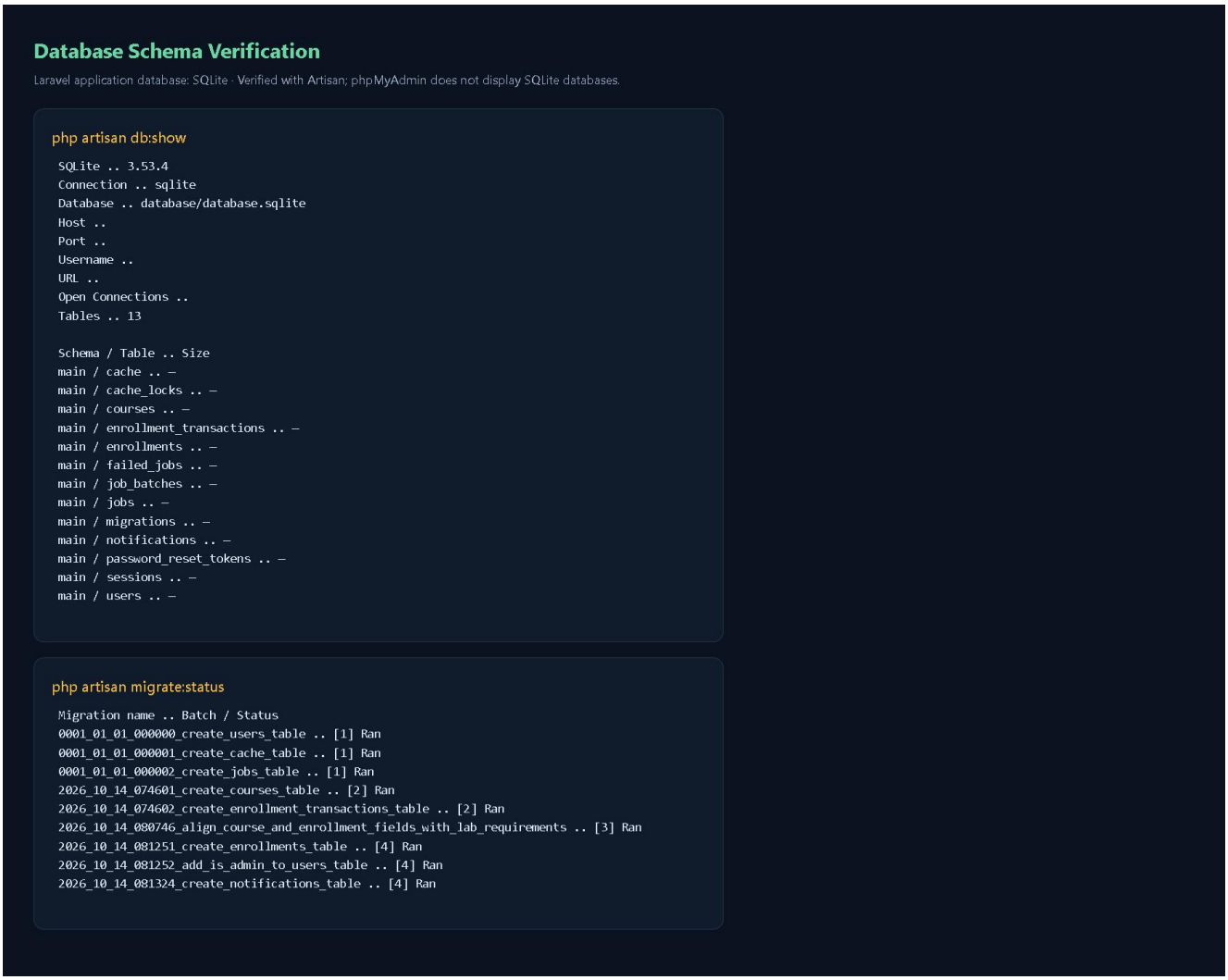
Screenshot 9: Edit course

Admin course editing and slot-threshold fields.



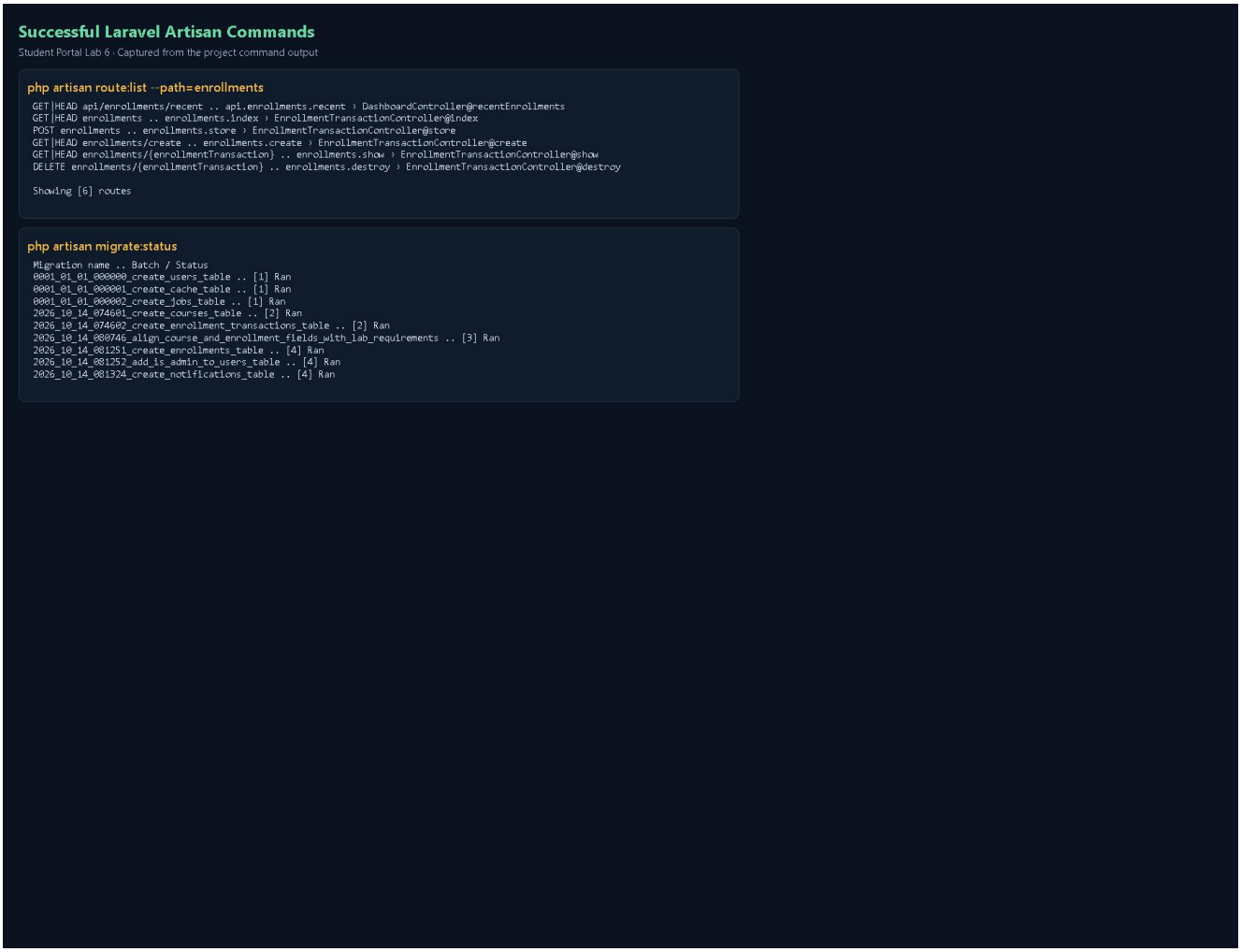
Screenshot 10: Database and migrations

SQLite schema and migration status verified with Laravel Artisan; substituted for phpMyAdmin because the app uses SQLite.



Screenshot 12: Successful Artisan commands

Enrollment route list and all database migrations marked as applied.



Screenshot 11: GitHub repository

PENDING - No GitHub destination URL was provided. The project has not been pushed, so this report does not present a local copy as a GitHub screenshot. Add the genuine repository screenshot after the remote is supplied and the push succeeds.